Reinforcement Learning for Tool-Augmented Vision-Language Models for Image-to-Image Manipulation

Alexis Bollengier
alexis.bollengier@epfl.ch

Badr Al Mahouri
badr.almahouri@epfl.ch

Anthony Richelle
anthony.richelle@epflch

***Abstract*—Vision-language models have strong capabilities in visual understanding, yet their ability to perform interactive image manipulation remains limited. We present a reinforcement learning framework that enables multimodal agents to learn tool usage for image-to-image transformations. Building on the VERL (Volcano Engine Reinforcement Learning for LLMs) library and extending prior work on mathematical and geometric reasoning tasks, we develop a training pipeline that teaches VLMs to interact with external image processing tools through Group Relative Policy Optimization (GRPO). We introduce a "render-and-obscure" data generation paradigm that creates synthetic training samples where ground truth information is intentionally inaccessible without proper tool usage, establishing a causal link between tool calls and task success, the model is also trained to use annotation tools. Our approach combines programmatic dataset generation with tool-aware reward functions. We demonstrate the effectiveness of this framework across multiple image manipulation tasks including bounding box localization, geometric transformations (flip and rotation), image de-blurring, and line drawing. Evaluation on both synthetic data and the real-world RefCOCO dataset.**

I. Introduction

VLMs extend LLMs' capabilities to multimodal understanding, enabling models to process and reason about visual inputs. However, most existing systems remain limited to passive visual understanding, they can describe and answer questions about images but cannot actively manipulate or transform them to extract obscured information or perform spatial operations. Inspired by ToolGym, which demonstrates the value of training agents to leverage external utilities, we explore how reinforcement learning (RL) can enable vision-language models to learn image-to-image tool usage. This project builds on the Volcano Engine Reinforcement Learning for LLMs (VERL)[1] framework, an open-source library designed for fine-tuning large language models on complex tasks using reinforcement learning. A. Initially, models have been trained on:

- GSM8K dataset [2]: 8,500 grade school math problems requiring basic arithmetic operations. The RL procedure teaches models to produce answers in a specific format (using #### markers) while ensuring numerical correctness.
- Geo3K dataset [3]: 3,000 geometry problems that require processing visual inputs of geometric diagrams. This work established initial patterns for handling image inputs within the VERL framework.

The big advantage of RL is that it doesn't require a smooth loss function; simple $0-1$ loss functions can be used, allowing for a much wider range of tasks. This also allows for more complex tasks, where it is hard to evaluate anything other than the final answer. The two main RL algorithms used in this project is Proximal Policy Optimization (PPO), which was first introduced with the GSM8K dataset (see [2]), and Group Relative Policy Optimization (GRPO) [4]. Let's first start with PPO. It works as follows :

- Works with an actor that generates the next token at each step and a critic model (also called value function) that estimates the expected future reward of a certain token at each step.
- After a response is completed and the reward is obtained, an advantage is computed, which is the difference between the observed return and the critic's estimate.
- This advantage is used to update both the critic and the actor models. The actor is updated so that tokens with a positive advantage are more likely and those with a negative advantage are less likely.
- The critic model is updated to better predict the value of each token in a certain context.

GRPO is a training method that has been introduced by DeepSeek to improve the performances of PPO. It does not use a critical model; instead, it does the following :

- For a given problem, the model generates multiple possible solutions, forming a group of outputs, and each solution is assessed separately.
- The average reward of the group serves as a baseline; the model is then updated to make better-than-average solutions more likely and discourage less-than-average solutions.
- This approach is usually more resource efficient by reducing the computational overhead due to the critic model. It also improves the memory usage.

II. Experimental Setup

A very time consuming part of the project was setting up the environment in which the RL would take place. The RL procedure requires a lot of inference and computation and thus has to be run on remote clusters. Some Python libraries need to be compiled with the cluster in mind to run efficiently (notably PyTorch), which is why custom wheels are used. The system being Linux Arch64, for which many Python libraries do not release all versions, means that manual compatibility matching is sometimes required to ensure the environment works, as some libraries have conflicting requirements. The GSM8K fine-tuning was done on Qwen2.5-0.5B using PPO. All image-based tasks use the Qwen2.5-VL-3B-Instruct model due to its vision-language capabilities, and it is fine-tuned using GRPO. Rewards are task-specific and designed for use with tools. Training is performed using GPU GH200 clusters with distributed data parallelism, and the SGLang framework generates multi-turn

rollouts with sequential tool calls. These clusters are the clusters from Alps (Clariden) [5].

III. Synthetic & Real-World Data

To incorporate tool usage into RL, we develop a programmatic data generation pipeline based on the "render-and-obscure" paradigm. Scenes are rendered with known latent attributes that are then obscured in a way that is specific to the target tool. This establishes a causal relationship whereby the ground truth cannot be derived from pixel data alone, meaning the agent must use the correct vision tool to determine the actionable state [1].

Each training sample is constructed around a fixed schema designed to support tool-based reinforcement learning. The dataset must contain the following five components:

1) **Prompt and Visual State:** A structured instruction sequence containing the `<image>` token and the initial visual input, in which the relevant information is intentionally inaccessible.
2) **Ground Truth Answer:** The latent correct answer associated with the image, typically stored in a normalized format such as `<boxed>` tags to enable reliable extraction during reward computation.
3) **Expected Tool Calls:** An explicit specification of the required tool interactions, including tool names and parameters, used to verify that the agent follows the intended action sequence.
4) **Oracle Metadata:** Hidden supervision signals stored in the `reward_model` structure, allowing the reward function to evaluate both the correctness of the final answer and the validity of the intermediate decision process.
5) **Data Serialization:** All samples are serialized using the Apache Parquet format via PyArrow. This choice is required to support deeply data structures, particularly for the `prompt` and `reward_model` fields.

A. Bounding Box

The model is trained on bounding boxes problems, the goal was for it to put a bounding box around an object or a set of objects and outline what was inside of it. Here the dataset consists in drawing bounding boxes around either labeled shapes or text and having to tell what color or what shape it is. The hope is that the model learns both spatial representation (mapping between the vision tokens and the pixel coordinates) and the different types of shapes and colors. The words are taken out of a 10k word list [6]. An example is drawn on Figure 1, the question could be, for example : *"Find the shape labeled F3 and draw a bounding box around it. What color is it?"* and for Figure 2: *"Where is the human with a red shirt? "*

The reward for this task is computed as $0.6 \cdot \text{IoU} + 0.4 \cdot G$, where G is the answer for the question and with B_{pred}, the predicted bounding box and B_{gt} the ground truth bounding box, IoU is :

$$\text{IoU} = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|} \quad \text{IoU} \in [0, 1] \tag{1}$$

[1]Bounding box and cropping tools operate slightly differently since their main goal is to teach the model spatial representation, the link between pixel coordinates and vision tokens.

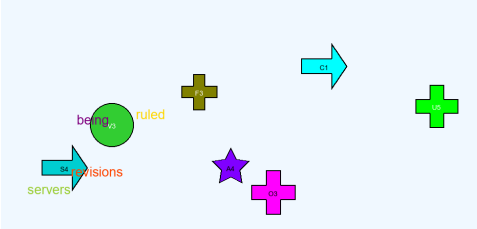

Figure 1. Bounding box training image example

Figure 2. RefCOCO training images

B. Flip and Rotation Tools

The dataset consists of text rendered on randomly selected backgrounds and subsequently transformed using geometric operations. Each sample is modified by applying either a horizontal or vertical flip, a rotation by a specified angle in the range $[0°, 360°]$, or a combination of both as shown in 3.

The Image Flip and Image Rotate Tools enable the model to correct these orientation distortions. To solve the task, the model must infer the applied transformation, invoke the appropriate tool with the correct parameters (flip direction or rotation angle), and then extract the recovered text from the transformed image. This process requires the model to reason explicitly about the image geometry before producing the final answer.

Correct tool usage is enforced through a rule-based reward function. A positive reward is assigned only if the model issues a valid tool call, the execution succeeds, and the final answer matches the ground truth within `<boxed>` tags. The reward is binary (1.0 for success, 0.0 otherwise).

C. Image Unblur Tools

An additional tool, which was not in the initial Tool Gym pipeline, was implemented : The Image Unblur Tool. It uses sharpening, unsharp masking, and edge enhancement operations to increase local contrast and enhance intensity transitions, making contours and fine details more visible, allowing the agent to modify image quality before attempting to extract information from visual inputs. An example training image is displayed on Figure 4, the prompt in that case could be : *"This image contains a hidden secret code that has been blurred. What is the secret code?"*

The reward function is used to evaluate agent behavior: a

Figure 3. Flipped and Rotated training image example

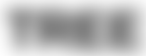

Figure 4. Blurred training image example

positive reward is assigned only if the agent issues a valid tool call for the input image and prompt, and if the final predicted answer matches the ground truth. Answers produced without tool usage, or following invalid tool calls, receive no reward. This design ensures that the agent learns to rely on explicit image manipulation rather than bypassing the tools.

D. Line Drawing Tool

We train the model to draw lines connecting specified points in images. The dataset comprises synthetic scenes containing points positioned at randomized coordinates. Natural language instructions specify connection targets using color or positional references (e.g. "connect the red point to the blue point").

The reward function evaluates coordinate accuracy through exponential decay based on endpoint precision:

$$r_{\text{line}} = \exp\left(-\frac{d_{\text{avg}}}{\tau}\right) \tag{2}$$

where d_{avg} is the average Euclidean distance between predicted and ground-truth line endpoints, and τ is a tolerance parameter.

E. Real-World Dataset

The RefCOCO dataset is built from real images taken from the MS COCO dataset. Each image is paired with a short natural language description that consists in labels and bounding boxes on the different elements in the picture, for example : "person" or "red car". The goal is to locate the described object by predicting its bounding box. If multiple objects have the same labels, locating one is deemed enough. RefCOCO provides bounding box annotations making it well-suited for object localization and spatial reasoning tasks grounded in natural language.

IV. Results

Figure 5 shows the evolution of the training and validation rewards for the bounding box task. At the beginning of training, the average reward is low (around 0.2), indicating that the model initially fails to reliably localize the relevant regions and produce correctly formatted call-ins for the bounding box tool.

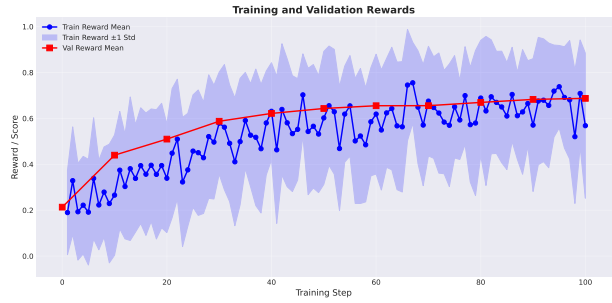

Figure 5. Training and validation reward curves for the bounding box task. The shaded area indicates one standard deviation around the training reward mean.

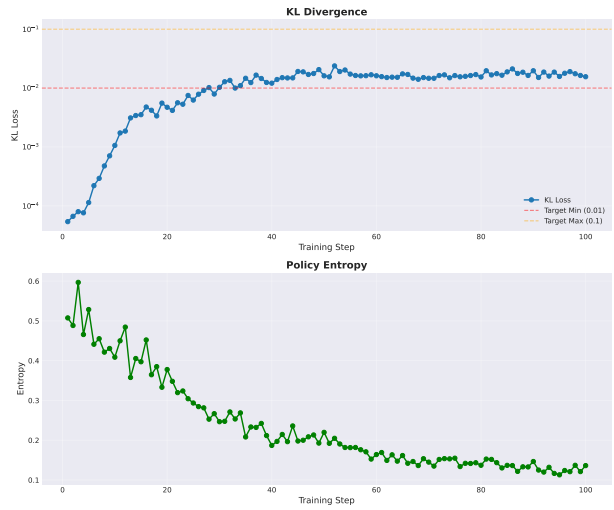

Figure 6. Policy regularization dynamics during bounding box training. **Top**: KL divergence between the updated policy and the reference policy, with target bounds. **Bottom**: policy entropy over training steps.

During the first training steps, the reward increases rapidly and stabilizes above 0.6, showing that the model quickly acquires a functional representation of the location of the object. The validation curve closely follows the training curve, suggesting good generalization and limited overfitting. The relatively large variance observed throughout training is expected, as the reward is based on the IoU, which is highly sensitive to small localization errors. If the model gives a bounding box with no intersection to the original one or fails to properly call the tool, the reward is automatically set to 0. This kind of failure still happens after the model is trained, albeit at a much lower rate.

Figure 6 provides additional insight into the optimization dynamics. The KL divergence initially increases from a very small value and stabilizes within the target range, indicating that policy updates remain controlled and do not diverge from the reference policy. This confirms that the regularization mechanism effectively constrains learning while still allowing meaningful policy improvement.

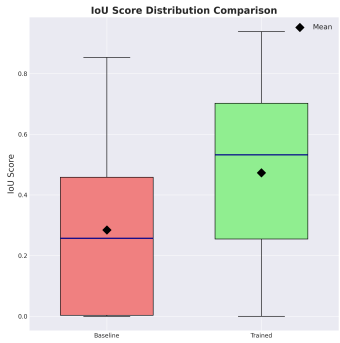

Figure 7. IoU improvement Generalization

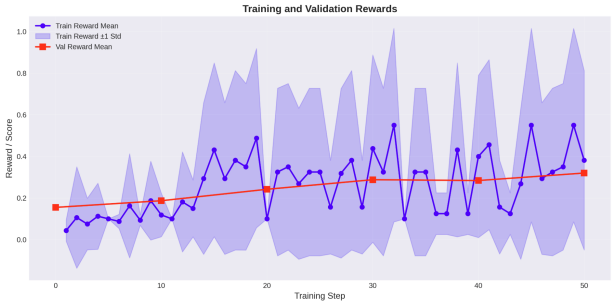

Figure 8. Training and validation reward curves for the blurring task. The shaded area indicates one standard deviation around the training reward mean.

At the same time, the policy entropy steadily decreases over training, reflecting a transition from exploratory behavior toward a more deterministic and confident tool usage. There is also no sudden drop in entropy, showing that the policy does not collapse either. This entropy reduction correlates with the stabilization of the reward curve and indicates that the model converges to consistent bounding box predictions rather than relying on stochastic exploration or a trivial policy.

The goal of any machine learning approach is generalization. To test this, once the model has been trained on the bounding box task, it is also evaluated on a cropping task, consisting in cropping the right object in a scene. To test if the training generalizes well the dataset is also made harder by adding a background with a pattern. Here, only the IoU matters for the score computation. This task is also made harder by adding a random background pattern. The results of this generalization test is highlighted in Figure 7, the mean IoU increases 66.1% and the median increases 106% showing considerable generalization of the training task to a new cropping task.

Overall, the joint analysis of reward, KL divergence, and entropy shows that bounding box training leads to stable reinforcement learning, controlled policy updates, and progressively more deterministic reasoning behavior. These properties are important for enabling reliable generalizable transfer to tasks such as zero-shot cropping. Others tools shows similar behaviour except for the blur and the draw line tool, that shows less good results as shown in Figure 8

V. Discussion

This project demonstrates that effective training of multimodal agents with image-based tools critically depends on careful reward design. Explicitly coupling task success with correct tool usage is crucial for obtaining stable reinforcement learning signals and preventing the model from bypassing visual tools. Continuous spatial rewards, such as IoU, provide informative rewards but introduce additional variance that must be controlled.

From a systems perspective, the Ray-based distributed execution framework enables efficient scaling of image processing operations, making multi-turn visual reinforcement learning feasible in practice. Furthermore, experiments on real-world datasets such as RefCOCO highlight the importance of grounding beyond synthetic data, revealing ambiguities and failure modes that do not appear in controlled environments.

Several challenges remain, including robust handling of bounding box edge cases, balancing reward granularity, and debugging multi-turn multimodal interactions. Additionally, the study is limited to a single, relatively small vision-language model and does not include ablation analyses of reward design choices.

VI. Conclusion & Future Work

In this project, we explored the use of reinforcement learning to train a vision-language model to interact with image manipulation tools. By combining tool-aware reward functions with controlled policy regularization, the model learned stable and reusable spatial reasoning behaviors.

The results show that training tools not only improve direct localization performance but also enable effective zero-shot transfer, for example, to cropping. This shows that tool-based supervision can induce generalizable visual skills. Overall, this work provides practical insights into reward design, system scalability, and evaluation strategies for multimodal reinforcement learning, and highlights promising directions for extending tool-augmented agents to more complex real-world visual tasks.

This framework could be extended to support more complex image manipulation tools and a wider range of real-world datasets. Scaling the approach to larger vision-language models and evaluating performance on higher-resolution images may further improve robustness and generalization. Additionally, enabling longer multi-step tool interactions could allow the model to tackle more advanced visual reasoning tasks.

VII. Ethical Risks

The primary risk involves individuals represented in real-world datasets like RefCOCO, who may be impacted by representational bias. If the model's "render-and-obscure" training paradigm is not sufficiently diverse, the tool-augmented agent could develop a "spatial bias," where its ability to localize, unblur, or transform images is significantly more accurate for certain demographic groups or environmental contexts than others. This could lead to a negative impact where the

model systematically fails to process information related to marginalized communities, potentially leading to exclusionary outcomes if deployed in automated surveillance, for example. We evaluate this risk by monitoring the variance in Intersection over Union (IoU) scores and policy entropy. High variance in these metrics during the RefCOCO evaluation reveals that real-world ambiguities often lead to "0-1 loss" failures that synthetic data does not capture. Future work must include reward-design ablation studies to ensure that the "causal link" between tool calls and task success remains ethically neutral across all visual domains.

While our current implementation utilizes the Qwen2.5-VL model, the objective remains the integration of Apertus, an open-weights model designed to provide a more transparent foundation for agentic tool use. Transitioning to Apertus would allow us to better mitigate the "black-box" nature of multimodal reasoning by leveraging its commitment to ethical data sourcing and architectural transparency.

REFERENCES

[1] G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu, "Hybridflow: A flexible and efficient rlhf framework," *arXiv preprint arXiv: 2409.19256*, 2024.

[2] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[3] P. Lu, R. Gong, S. Jiang, L. Qiu, S. Huang, X. Liang, and S.-C. Zhu, "Inter-gps interpretable geometry problem solving with formal language and symbolic reasoning," *The Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP 2021)*, 2021.

[4] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "Deepseekmath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024. [Online]. Available: https://arxiv.org/abs/2402.03300

[5] Swiss National Supercomputing Centre (CSCS), "Alps supercomputer," High-performance computing facility, 2024, access provided by CSCS.

[6] "Google-10000 english word list," https://apiacoa.org/publications/teaching/datasets/google-10000-english.txt, APIACOA, n.d., accessed: 2025-12-17.